

Dependability

A component provides **services** to clients.
To provide a service, it may depend on other components.
A component **C** depends on **C*** if C works correctly only when C* also works correctly.

Dependability Requirements	
Requirement	Meaning
Availability	The system is ready to use
Reliability	The service continues without failure
Safety	Very low chance of serious failures
Maintainability	Failed system can be repaired easily

Reliability versus Availability

Reliability

Reliability **R(t)** means the probability that a component works correctly during a time period [0, t].

Traditional Metrics

- MTTF — Mean Time To Failure:**
Average time until a component fails.
- MTTR — Mean Time To Repair:**
Average time needed to repair a component.
- MTBF — Mean Time Between Failures:**
Average time between two failures.
- MTBF = MTTF + MTTR**

Availability

Availability **A(t)** means the average fraction of time that component **C** is up and running.
Long-term availability:

A = MTTF / MTBF also can be written as **A = MTTF / (MTTF + MTTR)**

Reliability and availability are meaningful only if we clearly define what a **failure** is.

Failure, Error, Fault		
Term	Meaning	Example
Failure	Component does not meet its specification	Crashed program
Error	A wrong internal state that can cause failure	Programming bug
Fault	The cause of an error	Sloppy programmer

Handling Faults

Main Types

Term	Meaning	Simple Example
Fault prevention	Prevent faults before they happen	Do not hire bad programmers
Fault tolerance	Hide or mask the effect of a fault	Use two independent programmers/components
Fault removal	Reduce or remove existing faults	Fix bugs or remove bad code
Fault forecasting	Estimate future faults and their effects	Predict possible problems

Failure Models

Types of Failures

Failure Type	Meaning	
Crash failure	Server stops, but it worked correctly before stopping	
Omission failure	Server fails to send or receive messages	
→ Receive omission	Server fails to receive incoming messages	
→ Send omission	Server fails to send outgoing messages	
Timing failure	Server response is outside the expected time interval	
Response failure	Server gives an incorrect response	
→ Value failure	The response value is wrong	
→ State-transition failure	Server moves through the wrong control flow	
Arbitrary failure	Server may behave randomly or unpredictably	

Dependability versus Security

Omission vs Commission

Arbitrary failures can sometimes be **malicious**.

So, we distinguish two failure types:

Omission failure:

A component does not do an action that it should do.

Commission failure:

A component does an action that it should not do.

Observation

If failures are deliberate, they are usually security problems.

But in practice, it can be hard to know if a failure is deliberate or accidental.

Halting Failures

Scenario

Component C no longer sees any activity from component C*.

This may look like a **halting failure**.

But it can be hard to know if it is:

- a crash failure
- an omission failure
- a timing failure

Asynchronous vs Synchronous Systems

Asynchronous system:

There are no fixed limits for process speed or message delay.

So, crash failures cannot be detected reliably.

Synchronous system:

Process speed and message delay have known limits.

So, omission and timing failures can be detected reliably.

Partially synchronous system:

Most of the time, the system behaves like a synchronous system.

So, crash failures can usually be detected reliably.

Halting Failures	
Halting Type	Meaning
Fail-stop	Crashes and is clearly detected.
Fail-noisy	Crashes and is eventually detected.
Fail-silent	No response; cause is unknown.
Fail-safe	Wrong behavior, but no harm.
Fail-arbitrary	Random or malicious behavior.

Redundancy for Failure Masking

Information redundancy:

Add extra bits to detect or fix data errors.

Time redundancy:

Retry the action if something goes wrong.

Physical redundancy:

Use extra components so the system keeps working.

Process Resilience

Process resilience protects the system from faulty processes by using **process replication**.

Multiple processes are organized into a **process group**.

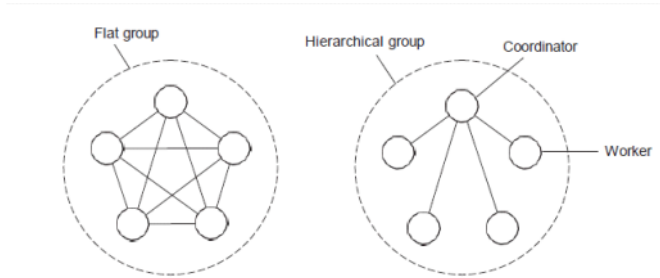
Group Types

Flat group:

All processes are equal and communicate with each other.

Hierarchical group:

One process is the **coordinator**, and the others are **workers**.



Groups and Failure Masking

k-Fault Tolerant Group

A group is **k-fault tolerant** if it can hide **k simultaneous member failures**.

Group Size

For halting failures:

We need **$k + 1$ members**.

One correct member is enough because failed members do not give wrong results.

For arbitrary failures:

We need **$2k + 1$ members**. k kadar fail member majority için $k+1$ lazım $2k+1$

The correct result is chosen by **majority vote**.

Assumptions

- All members are identical.
- All members process commands in the same order.

Consensus

Prerequisite(ön koşul)

In a fault-tolerant process group, every correct process must execute the **same commands in the same order**.

If processes execute commands in different orders, they may produce different results.

Reformulation

So, the problem becomes simpler:

Before executing the next command, all correct processes must **agree on the same command**. This agreement is called consensus.

Realistic Consensus: Paxos

Assumptions

Paxos works with realistic and weak assumptions:

- The system can be partially synchronous or even asynchronous.
- Messages may be lost, duplicated, or reordered.
- Corrupted messages can be detected and ignored.
- Operations are deterministic.
- Processes may crash, but they do not behave arbitrarily.
- Processes do not collude.

Paxos Essentials

Starting Point

We start with a client-server system with one **primary server**.
To make it more reliable, we add a **backup server**.
The primary gives each command a unique **sequence number**.
This ensures that both servers execute commands in the same order.
In Paxos, the primary server is called the **leader**.

Control Messages

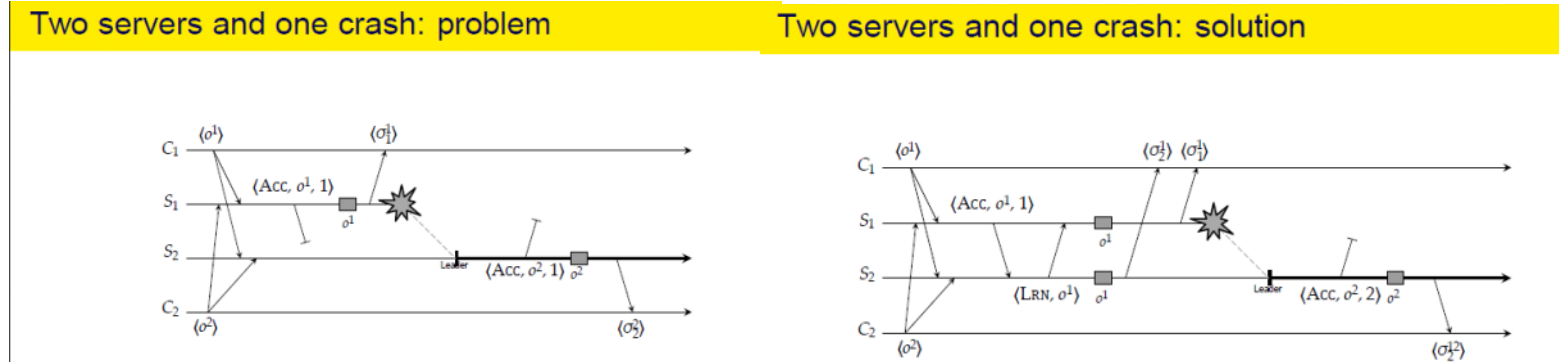
Actual commands can be restored later.
So, Paxos mainly focuses on **control messages**.

Handling Lost Messages

ACCEPT(o, t):
Leader sends operation **o** with timestamp **t** to backups.

LEARN(o, t):
Backup confirms that it learned **o** with timestamp **t**.
If the message is lost, the leader sends **ACCEPT(o, t)** again with the same timestamp.

Short meaning:
Lost messages are retransmitted without changing the operation order.



Solution:
Do not execute an operation until the backup has learned it.

Short meaning:
The primary should not act alone. The backup must know the operation first, so the system can continue correctly after a crash.

Paxos: Fundamental Rule

General Rule

A server cannot execute an operation until it receives **LEARN(o)** from all other nonfaulty servers.

Failure Detection

Practice

Reliable failure detection is almost impossible in practice.

A common solution is to use **timeouts**.

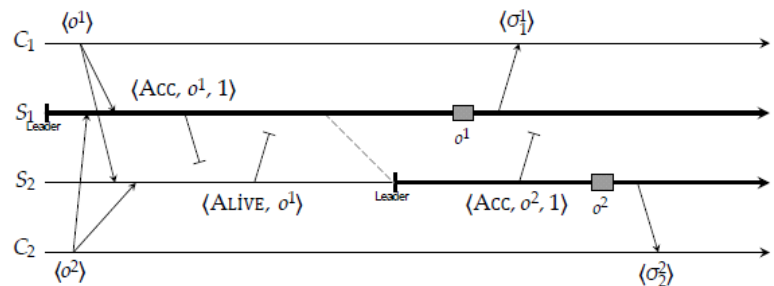
If a server does not respond in time, it may be considered failed.

Important Point

A detected failure may be **false**.

The server may not be crashed.

It may only be slow or delayed.



Required Number of Servers

Observation

Paxos needs at least **three servers**.

Adapted Fundamental Rule

With three servers, a server can execute operation o after receiving at least **one other** **LEARN(o)** message.

Because then at least **two servers** know the operation.

Short Meaning

In Paxos, an operation should be executed only when a **majority** of servers know it.

For 3 servers:

majority = 2 servers

So, one server + one other LEARN message is enough.